

Lecture 5

The Decorator Pattern

Learning Objectives

- Understand the Decorator Pattern
- Understand the key components of the Decorator Pattern

Senior – Lecturer

Almas Ospanov



HFDP, Chapter 3: The Decorator Pattern: Decorating Objects. Part I

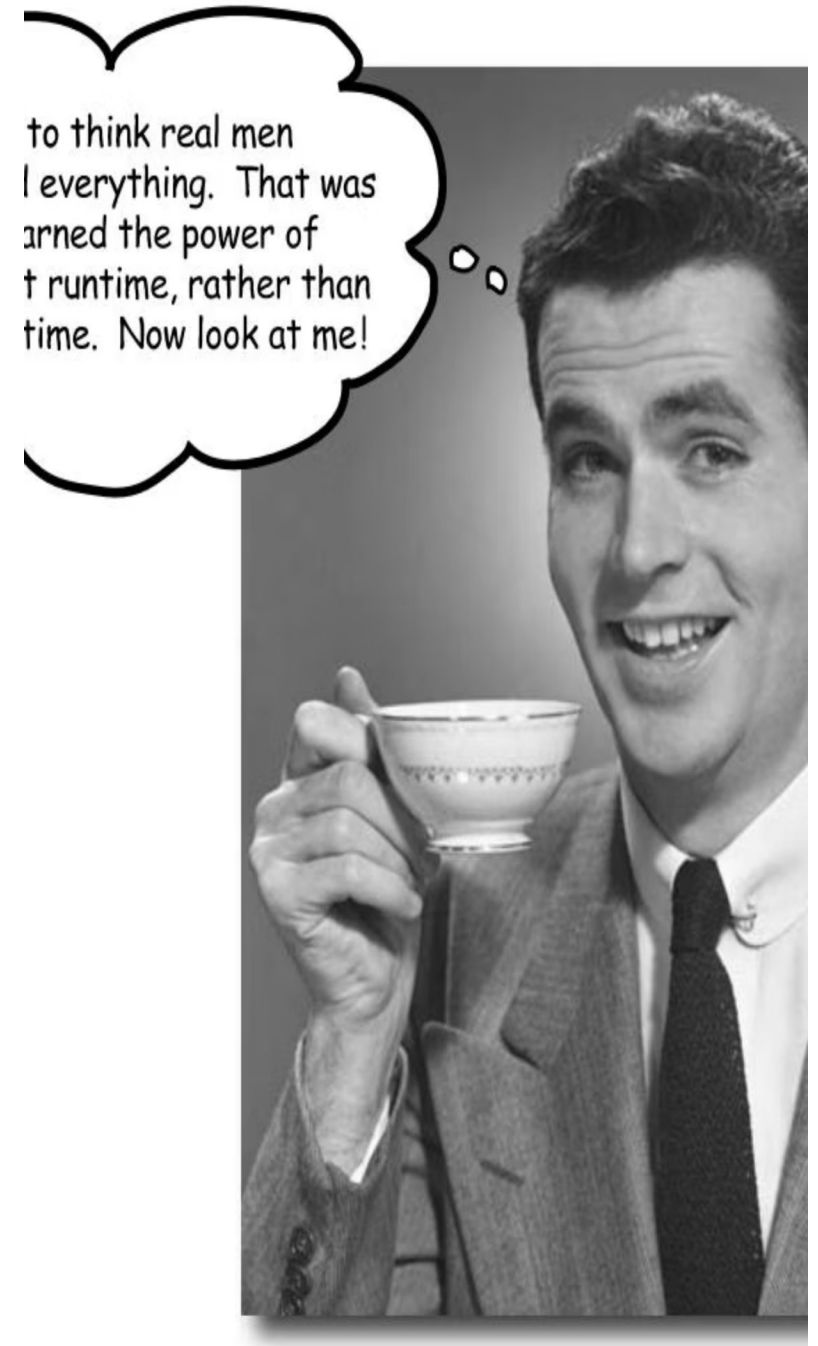
The Decorator Pattern is a powerful tool in object-oriented design. It enhances object functionality dynamically without altering their core structure.

We'll explore its principles, practical applications, and how it offers a flexible alternative to traditional inheritance. Get ready to "decorate" your classes at runtime and build more robust, extensible software systems.

Design Eye for the Inheritance Guy

Let's re-examine the common difficulties of over-relying on inheritance in software design. While inheritance is a fundamental concept, its misuse can lead to rigid, hard-to-maintain codebases.

It's important to know how to "decorate" your classes at runtime using a form of object composition. This technique allows you to add new responsibilities to existing objects without making any code changes to the underlying classes. This flexibility is crucial for building systems that can easily adapt to changing requirements.



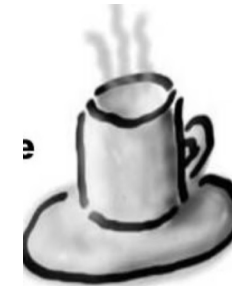
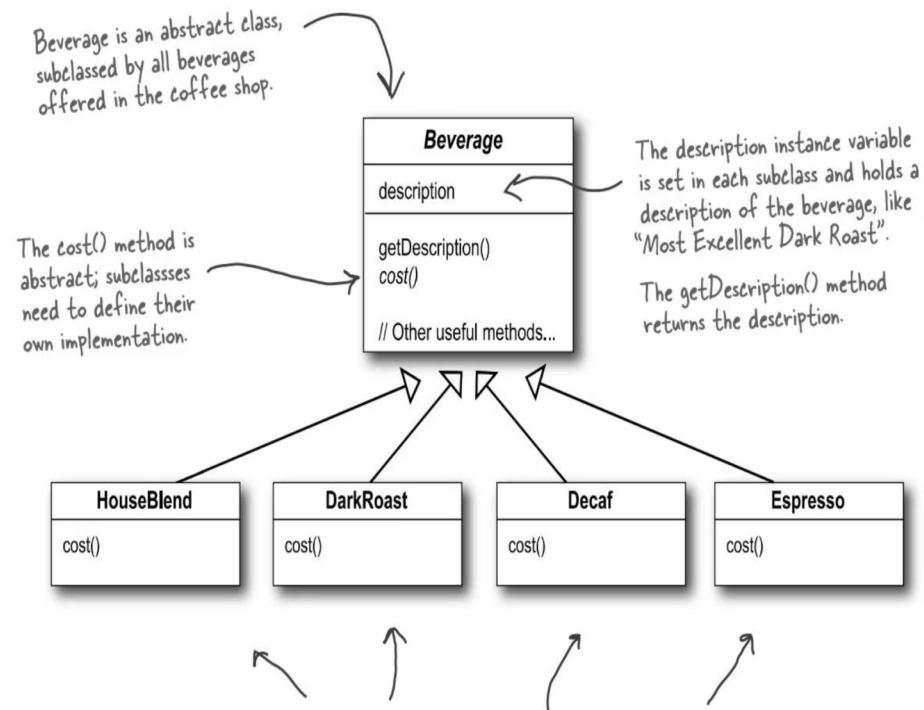
to think real men
l everything. That was
arned the power of
t runtime, rather than
time. Now look at me!

The Starbuzz Coffee Challenge

Starbuzz Coffee, a rapidly expanding coffee chain, is facing a common software design challenge: updating their ordering system to keep pace with their diverse beverage and condiment offerings.

Initially, their system was designed with a straightforward inheritance hierarchy for beverages, where each subclass implemented a `cost()` method to return the beverage's price.

However, Starbuzz also offers various condiments like steamed milk, soy, mocha, and whipped milk, each with an additional charge. Integrating these into the existing design proved problematic.



Their initial attempt to incorporate condiments led to an explosion of subclasses, creating a complex and inflexible system. This scenario highlights the limitations of inheritance when dealing with a combinatorial explosion of features.

The Impact of Change: Why Inheritance Fails Here

The initial inheritance-based design for Starbuzz Coffee's ordering system quickly reveals its weaknesses when faced with evolving business requirements.

Price Changes

Any change in condiment pricing would necessitate altering existing code within numerous subclasses, violating the principle of being "closed for modification."

New Condiments

Adding a new condiment would require creating new methods and modifying the `cost()` method in the superclass, leading to further code changes and potential bugs.

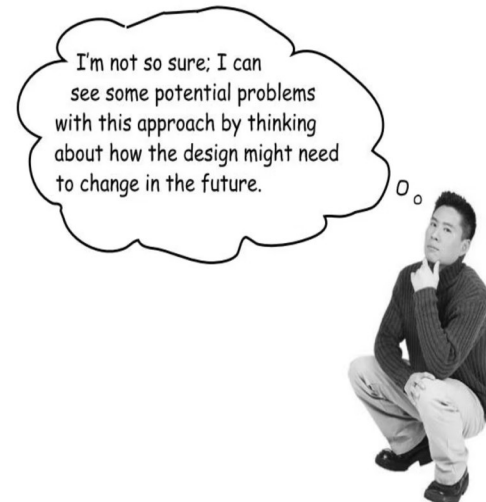
New Beverages

Introducing new beverages (e.g., iced tea) might result in inheriting inappropriate condiment methods (e.g., `hasWhip()`), leading to illogical and confusing class structures.

Combinatorial Explosion

Handling multiple instances of the same condiment (e.g., double mocha) becomes incredibly complex, requiring an exponential increase in subclasses to cover all possible combinations.

This rigid design, as illustrated by the class diagrams, is highly susceptible to the "impact of change," making maintenance and feature expansion a nightmare.



The Open-Closed Principle

This principle is fundamental to building robust and maintainable software systems. It states:

"Classes should be open for extension, but closed for modification."

Open for Extension

This means that the behavior of the module can be extended to meet new requirements. You should be able to add new functionality without altering the existing code.



Closed for Modification

Once a module has been developed and tested, its source code should not be changed. This prevents introducing new bugs into existing, working code.



Achieving both "open for extension" and "closed for modification" might seem contradictory, but it's the cornerstone of flexible and resilient designs. Our goal is to allow classes to incorporate new behavior without modifying existing code, leading to designs that are resilient to change and flexible enough to take on new functionality.

Implementation of the Decorator Pattern

Given the limitations of inheritance for handling beverage and condiment pricing, the Decorator Pattern offers a flexible and elegant solution. Instead of subclassing, we "decorate" a beverage object with condiments at runtime.

1 Start with a Base Beverage

Begin with a concrete beverage object, such as a `DarkRoast`.

2 Decorate with Condiments

Wrap the beverage object with condiment decorators, for example, a `Mocha` object, then a `Whip` object. Each decorator adds its own behavior and cost.

3 Delegate Cost Calculation

When `cost()` is called on the outermost decorator, it delegates the cost calculation to the object it decorates, adding its own cost in the process. This chain of delegation calculates the total price.

This approach treats a decorated object (e.g., `DarkRoast` with `Mocha` and `Whip`) as still being of the original beverage type, allowing seamless interaction with existing code.



ricing
iss
that isn't

decorate"
wants a

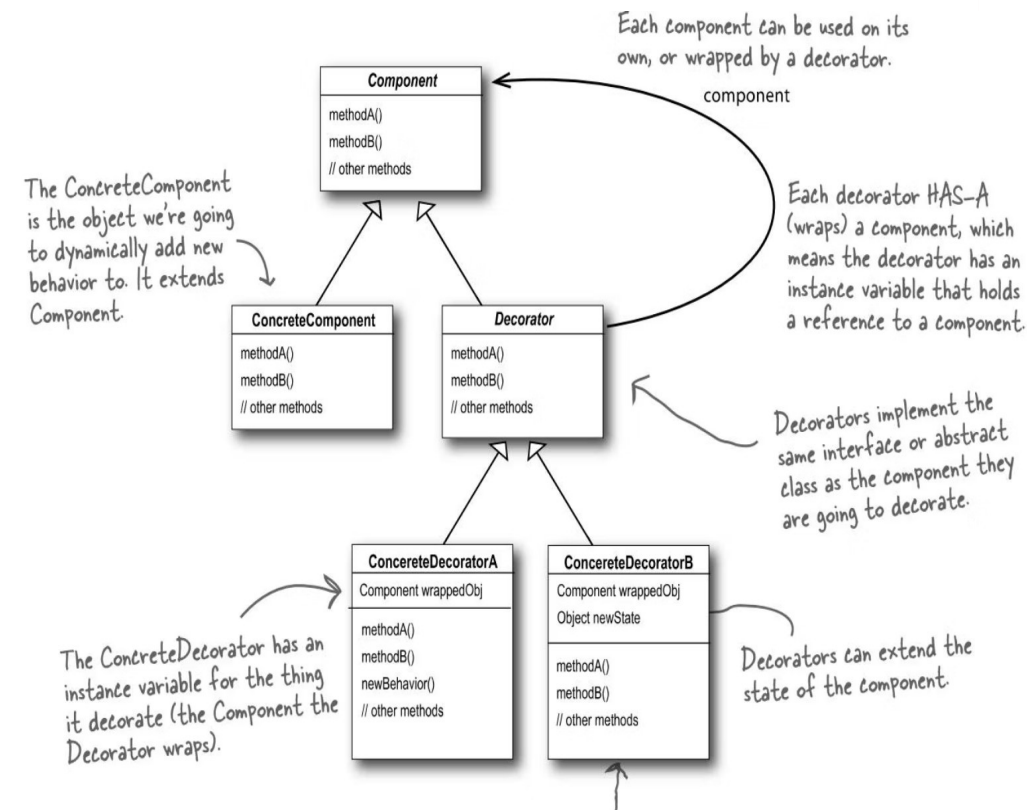
The Decorator Pattern Defined

The Decorator Pattern provides a structural solution to dynamically adding responsibilities to objects.

"The Decorator Pattern attaches additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality."

This pattern allows you to wrap objects with other objects that provide additional behavior, without modifying the original object's code. Key characteristics include:

- Decorators share the same supertype as the objects they decorate, ensuring type compatibility.
- Multiple decorators can be used to wrap an object, allowing for flexible combinations of features.
- Decorators add their behavior before and/or after delegating to the wrapped object.
- Objects can be decorated dynamically at runtime, providing immense flexibility.



The class diagram illustrates how a **Component** (e.g., **Beverage**) can be wrapped by a **Decorator** (e.g., **CondimentDecorator**), which also implements the **Component** interface. Concrete components (e.g., **Espresso**) and concrete decorators (e.g., **Mocha**) extend these.

Inheritance vs. Composition in Decorator

A common point of confusion arises regarding the use of inheritance within the Decorator Pattern, especially when the decorator extends the component's abstract class.

“

Mary's Confusion

"I thought we weren't going to use inheritance in this pattern, but rather we were going to rely on composition instead. The `CondimentDecorator` is extending the `Beverage` class. That's inheritance, right?"

“

Sue's Clarification

"True. The point is that it's vital that the decorators have the same type as the objects they are going to decorate. So here we're using inheritance to achieve the type matching, but we aren't using inheritance to get behavior."

“

“

The key takeaway is that decorators use inheritance for type compatibility (ensuring they can stand in place of the component) and composition for behavior extension. This allows for dynamic behavior modification at runtime, offering greater flexibility than static inheritance.

- **Behavior through Composition:** New behavior is acquired by composing objects together, not by inheriting it from a superclass.
- **Dynamic Flexibility:** Object composition allows for mixing and matching decorators in any way desired, at runtime.
- **Extensibility:** New decorators can be implemented at any time to add new behavior without modifying existing code.

Real-World Decorators: Java I/O

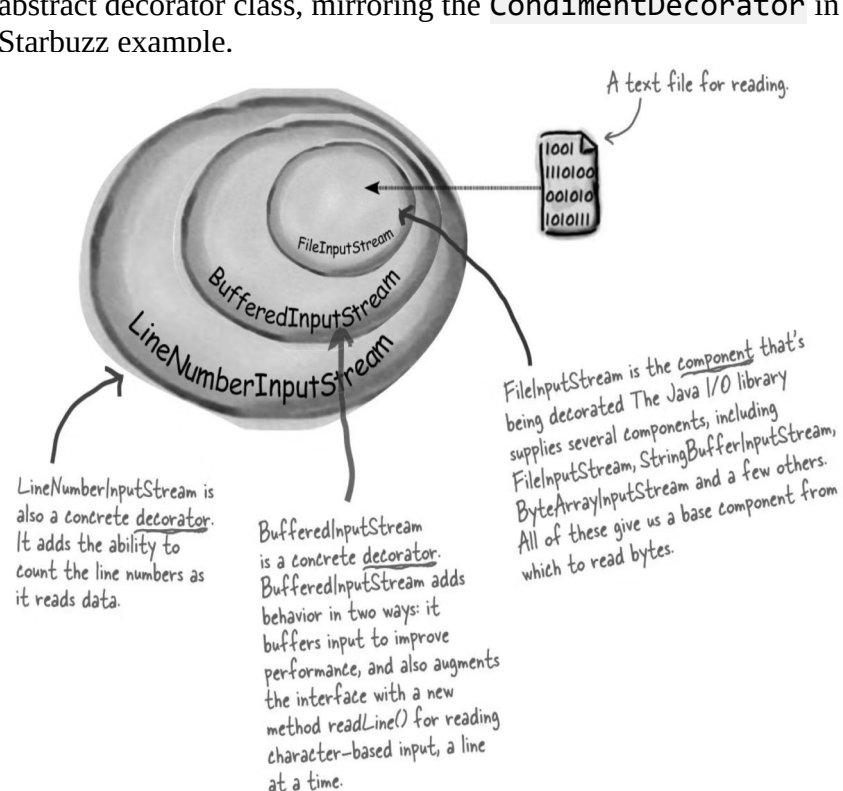
The Java I/O package is a prime example of the Decorator Pattern in action. While initially overwhelming, understanding Decorator makes its structure much clearer.

Decorating Input Streams

Consider reading data from a file. You start with a basic `FileInputStream`, and then wrap it with various decorators to add functionality:

- `BufferedInputStream` for efficient buffering.
- `LineNumberInputStream` to track line numbers.

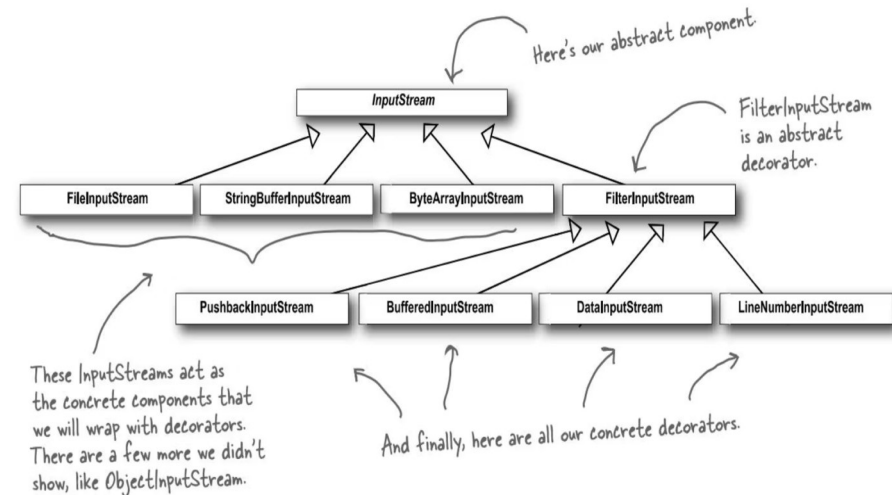
These decorators extend `FilterInputStream`, which acts as the abstract decorator class, mirroring the `CondimentDecorator` in our Starbuzz example.



Benefits and Downsides

This design allows for highly flexible and customizable I/O operations. You can combine different functionalities (buffering, character conversion, compression) by simply wrapping streams.

However, Java I/O also highlights a potential downside of the Decorator Pattern: it can lead to a large number of small classes, which might seem overwhelming at first glance. But with a clear understanding of the



Key Takeaways from Chapter 3, HFDP: The Decorator Pattern

The Decorator Pattern is a powerful design tool for extending object functionality dynamically.



Flexible Extension

Provides an alternative to subclassing for extending behavior, allowing new functionalities to be added at runtime without modifying existing code.



Composition & Delegation

Leverages composition and delegation to add new behaviors, where decorators wrap concrete components and delegate method calls, adding their own logic before or after.



Type Mirroring

Decorator classes mirror the type of the components they decorate, ensuring transparency to the client as long as the client relies on the abstract component type.

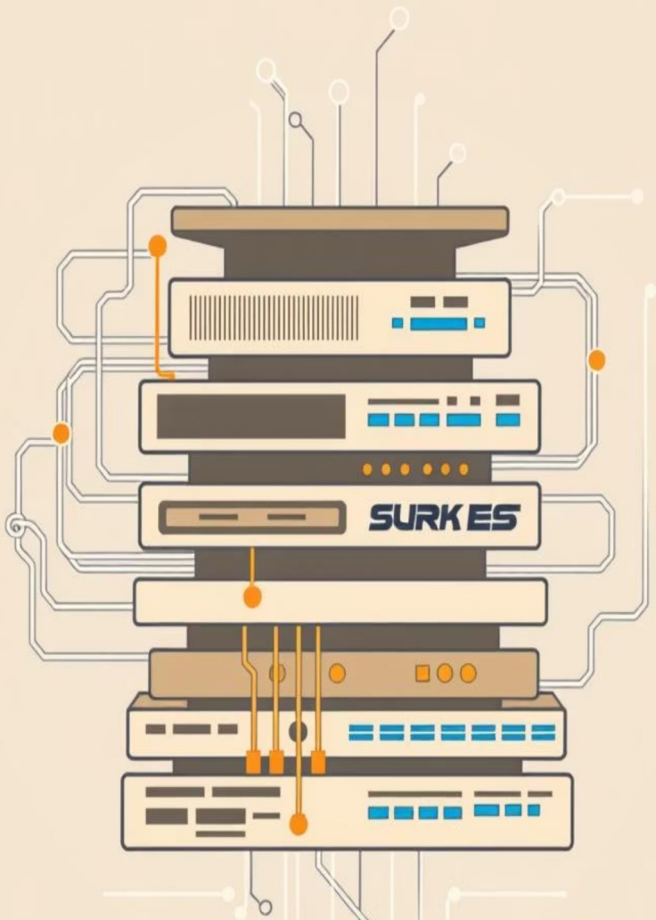


Potential Complexity

Can result in many small objects, potentially increasing design complexity, but this is often mitigated by using other patterns like Factory or Builder for instantiation.

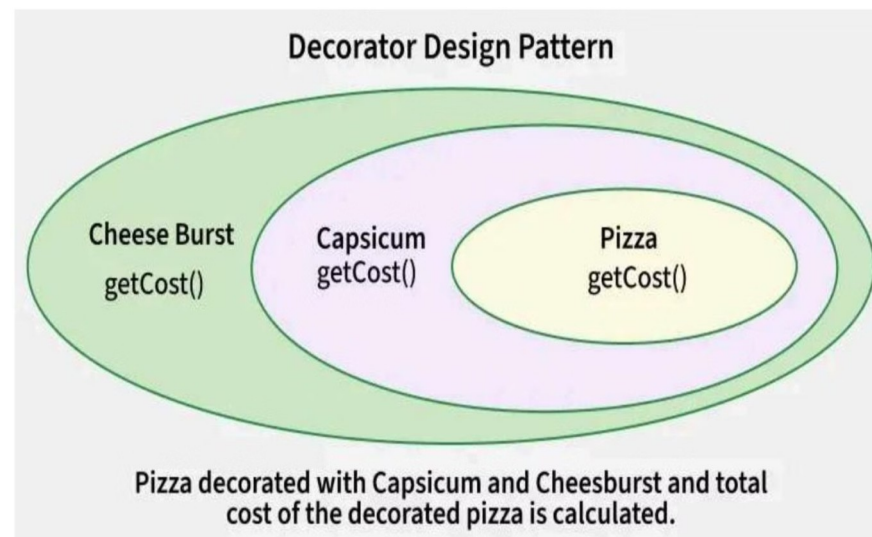
By understanding and applying the Decorator Pattern, you can create more flexible, maintainable, and extensible software designs that adhere to principles like the Open-Closed Principle.

The Decorator Design Pattern, Part II



The Decorator Design Pattern is a structural design pattern that allows you to dynamically add new behaviors or responsibilities to individual objects without altering their core structure. This pattern provides a flexible alternative to subclassing for extending functionality.

This pattern is particularly useful when you want to extend the functionality of an object without creating a large number of subclasses, which can lead to a complex and inflexible class hierarchy. Instead, decorators wrap the original object, adding new features while maintaining the original object's interface.



Source: [Geekforgeeks.org](https://www.geekforgeeks.org/)

The diagram above illustrates a Pizza (base component) wrapped with Capsicum and then Cheese Burst (decorators). Each decorator adds its own cost via `getCost()`, so the final price is the sum of the Pizza, Capsicum, and Cheese Burst costs. This demonstrates how the Decorator Pattern layers extra behavior dynamically.

Real-World Applications of the Decorator Pattern

The Decorator Design Pattern is widely used in various software applications to enhance flexibility and reusability. Here are some prominent real-world examples:

Video Streaming Platforms

In a video streaming platform, a base video content object can be decorated with features like subtitles, language preferences, video quality options (e.g., HD, 4K), and audio enhancements. Each of these features can be added or removed dynamically without modifying the core video object.

Text Processing Applications

Decorators are excellent for applying formatting or transformations to text. A `PlainText` object can be wrapped with a `BoldTextDecorator` or an `ItalicTextDecorator` to apply specific styles. This allows for flexible text styling without altering the original text content or creating numerous subclasses for each style combination.

Java I/O Package

The `java.io` package is a classic example. A `FileInputStream` (concrete component) can be wrapped by a `BufferedInputStream` to add buffering capabilities, or by a `DataInputStream` to add the ability to read primitive data types. Each wrapper adds functionality without changing the fundamental `FileInputStream`.

These examples highlight the pattern's utility in scenarios where optional features or behaviors need to be added to objects in a flexible and reusable manner, such as in text formatting or graphical user interfaces.

Key Components of the Decorator Pattern

Understanding the core components is crucial for implementing the Decorator Design Pattern effectively. Each component plays a distinct role in enabling dynamic behavior extension.

01

Component Interface

This interface defines the common operations that both the concrete components and the decorators must implement. It ensures that decorators can wrap components seamlessly, as they all adhere to the same contract.

03

Decorator (Abstract Wrapper)

This is an abstract class that also implements the Component Interface. It maintains a reference to a Component object (the decorated object) and delegates requests to it. It serves as the base class for all concrete decorators.

By separating these concerns, the Decorator Pattern promotes a highly modular and extensible design, allowing for easy addition of new features without modifying existing code.

02

Concrete Component

This is the core object that provides the basic, unadorned functionality. It implements the Component Interface and represents the object to which additional responsibilities will be attached.

04

Concrete Decorator

These are specific classes that extend the abstract Decorator. They add new responsibilities or modify existing ones before or after delegating the request to the wrapped component. Multiple concrete decorators can be stacked to add multiple functionalities.

Problem Statement: Coffee Shop Application

To illustrate the Decorator Design Pattern, consider the following problem:

Suppose we are building a coffee shop application where customers can order different types of coffee. Each coffee can have various optional add-ons such as milk, sugar, whipped cream, etc. We want to implement a system where we can dynamically add these add-ons to a coffee order without modifying the coffee classes themselves.

This scenario is a perfect fit for the Decorator Pattern because it allows us to:

- Add optional features (add-ons) to coffee orders dynamically.
- Avoid altering the core coffee classes, promoting the Open/Closed Principle.
- Ensure code flexibility, scalability, and maintainability.
- Easily introduce new add-ons and combine them with different types of coffee orders.

Without the Decorator Pattern, we might end up with an explosion of subclasses (e.g., `MilkCoffee`, `SugarCoffee`, `MilkSugarCoffee`), which quickly becomes unmanageable as the number of add-ons grows.

Code Breakdown: Component Interface (Coffee)

Let's break down the code implementation for our coffee shop application, starting with the fundamental Component Interface.

The Coffee Interface

This interface serves as the contract for all coffee types and their decorators. It declares two essential methods:

- `getDescription()`: Returns a string describing the coffee, including any added decorators.
- `getCost()`: Returns the total cost of the coffee, including the base cost and any decorator costs.

Both concrete coffee types and all decorators must implement this interface, ensuring a consistent API for clients.

This simple interface is the cornerstone of the Decorator Pattern, allowing for interchangeable components and flexible extensions.

```
public interface Coffee {    String  
    getDescription();    double getCost();}
```

Code Breakdown: Concrete Component & Decorator

Next, we examine the Concrete Component and the Abstract Decorator classes.

Concrete Component: PlainCoffee

The `PlainCoffee` class is our basic coffee without any add-ons. It implements the `Coffee` interface and provides the fundamental description and cost.

- `getDescription()` returns "Plain Coffee".
- `getCost()` returns a base cost, e.g., \$2.0.

C++ Java Python JavaScript

```
/*package whatever //do not write package name here */  
  
import java.io.*;  
  
class GFG {  
    public static void main (String[] args) {  
        System.out.println("GFG!");  
    }  
}
```

Abstract Decorator: CoffeeDecorator

The `CoffeeDecorator` is an abstract class that also implements the `Coffee` interface. It holds a reference to the `decoratedCoffee` object and delegates calls to its methods.

- It has a constructor that takes a `Coffee` object.
- Its `getDescription()` and `getCost()` methods simply call the corresponding methods on the `decoratedCoffee` object. Concrete decorators will override these to add their specific behavior

C++ Java Python JavaScript

```
// PlainCoffee.java  
public class PlainCoffee implements Coffee {  
    @Override  
    public String getDescription() {  
        return "Plain Coffee";  
    }  
}
```

This setup ensures that decorators can be chained, with each decorator adding its own layer of functionality while still maintaining the original object's interface.

Code Breakdown: Concrete Decorators

Finally, we implement the Concrete Decorators that add specific add-ons to our coffee.

MilkDecorator

This class extends `CoffeeDecorator` and adds milk to the coffee. It overrides the `getDescription()` and `getCost()` methods:

- `getDescription()`: Appends ", Milk" to the decorated coffee's description.
- `getCost()`: Adds the cost of milk (e.g., \$0.5) to the decorated coffee's cost.

```
public class MilkDecorator extends CoffeeDecorator
{
    public MilkDecorator(Coffee decoratedCoffee) {
        super(decoratedCoffee);
    }

    @Override    public String getDescription() {

return decoratedCoffee.getDescription() + ",
Milk";    }

@Override    public double getCost()
{
    return decoratedCoffee.getCost() + 0.5;
}}
```

SugarDecorator

Similar to `MilkDecorator`, this class adds sugar to the coffee. It also overrides the description and cost methods:

- `getDescription()`: Appends ", Sugar" to the decorated coffee's description.
- `getCost()`: Adds the cost of sugar (e.g., \$0.2) to the decorated coffee's cost.

```
public class SugarDecorator extends
CoffeeDecorator {    public SugarDecorator(Coffee
decoratedCoffee) {        super(decoratedCoffee);
    }

@Override    public String getDescription() {
        return decoratedCoffee.getDescription() +
", Sugar";    }

@Override    public double getCost()
{
    return decoratedCoffee.getCost() + 0.2;
}}
```

These concrete decorators demonstrate how new functionalities can be added incrementally and independently, allowing for flexible combinations of add-ons.

Advantages & Disadvantages of the Decorator Pattern

Like any design pattern, the Decorator Pattern comes with its own set of benefits and drawbacks that developers should consider.

Advantages

- Open-Closed Principle

Allows extending functionality without modifying existing code, promoting maintainability.

- Flexibility

Behaviors can be added or removed from objects at runtime, offering dynamic customization.

- Reusable Code

Decorators can be reused across different objects, reducing code duplication and promoting modularity.

- Composition over Inheritance

Avoids rigid class hierarchies by combining decorators, leading to more flexible designs.

- Clear Structure

Provides an organized way to add features, making the design easier to understand and manage.

Disadvantages

- Complexity

Too many nested decorators can make the code harder to debug and understand the flow of execution.

- Class Proliferation

Can lead to a large number of small decorator classes, which might increase the overall codebase size.

- Order Sensitivity

The behavior of the decorated object might depend on the specific order in which decorators are applied.

- Overuse Risk

If misused or applied unnecessarily, it can make the code overly complex without providing significant benefits.

- Language Limitations

In some programming languages, implementing decorators might be more cumbersome or less idiomatic.

Key Takeaways & Further Exploration from Part II

The Decorator Design Pattern is a powerful tool for extending object functionality dynamically. As computer engineering students, mastering such patterns is crucial for building robust, flexible, and maintainable software systems.



Dynamic Extension

Remember that decorators allow you to add responsibilities to objects at runtime, offering greater flexibility than static inheritance.



Composition over Inheritance

Embrace composition to build complex behaviors from simpler ones, avoiding the pitfalls of deep inheritance hierarchies.

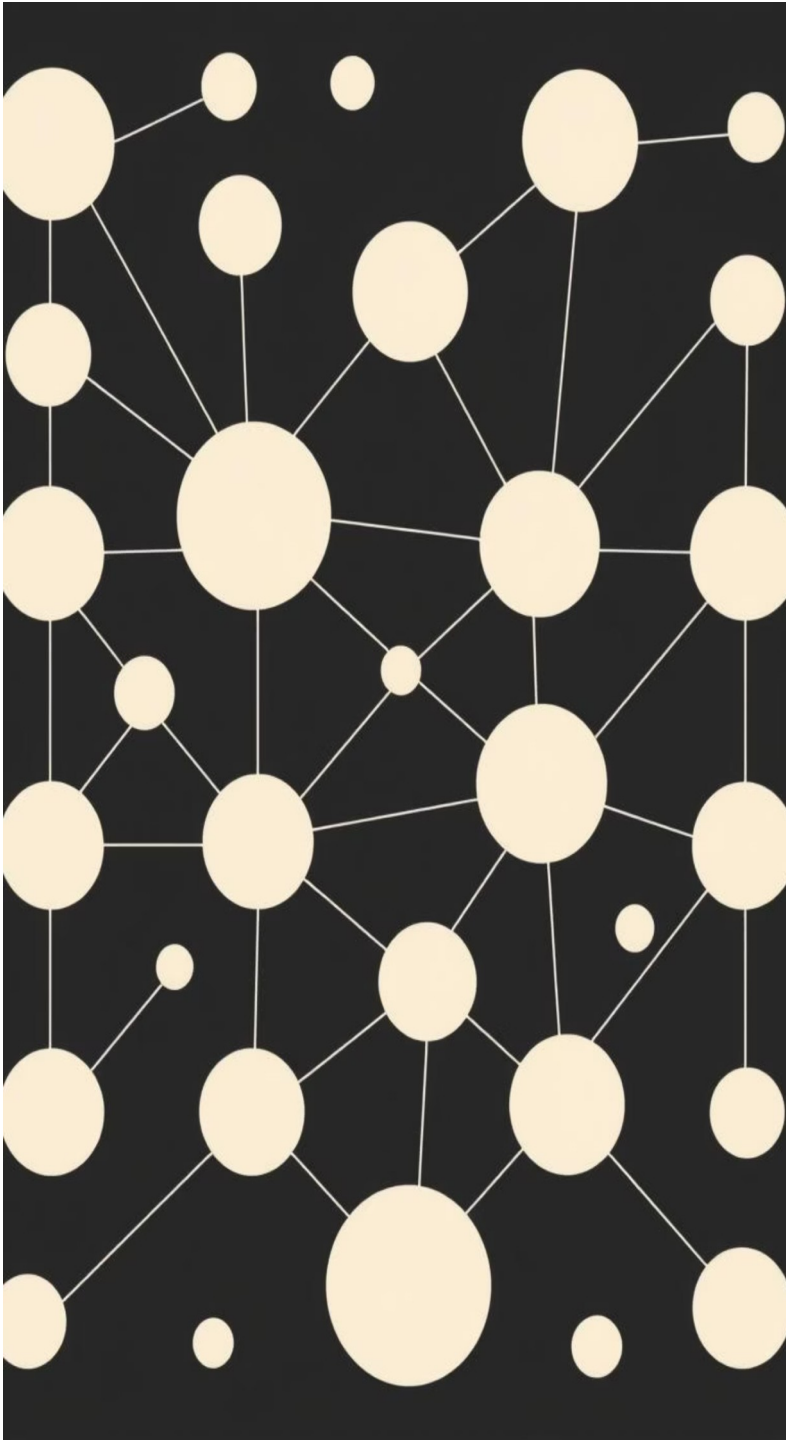


Practical Application

Look for opportunities to apply this pattern in your projects, especially when dealing with optional features or layered functionalities.

Further Exploration

To deepen your understanding, explore other structural design patterns and how they compare to the Decorator Pattern. Consider how this pattern integrates with other design principles and architectural styles.



The Decorator Design Pattern, Part III

Enhancing Object Behavior Dynamically

"Decorator is a structural design pattern that lets you attach new behaviors to objects by placing these objects inside special wrapper objects that contain the behaviors."

Also known as : Wrapper.

Source: refactoring.guru

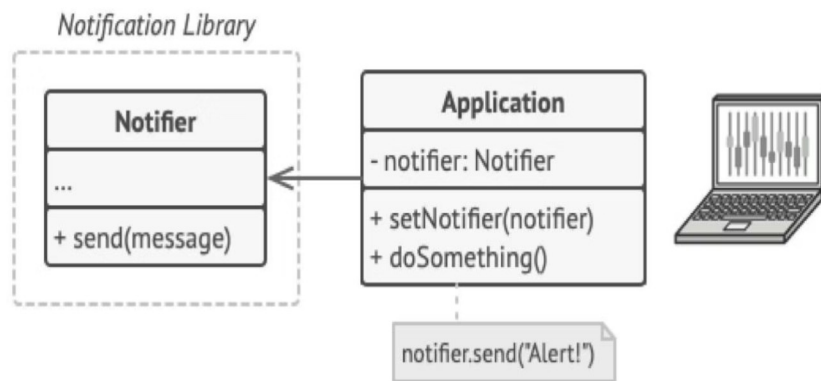
The Notification Problem

A notification library which lets other programs notify their users about important events.

Source: refactoring.guru

Initial Setup

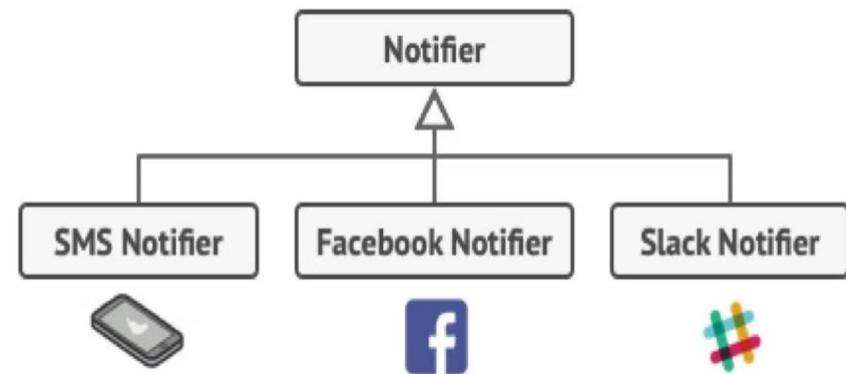
A **Notifier** class sends messages to emails. A third-party app uses it for important events.



An Application uses the notifier class to send notifications about events to a predefined set of emails.

Growing Demands

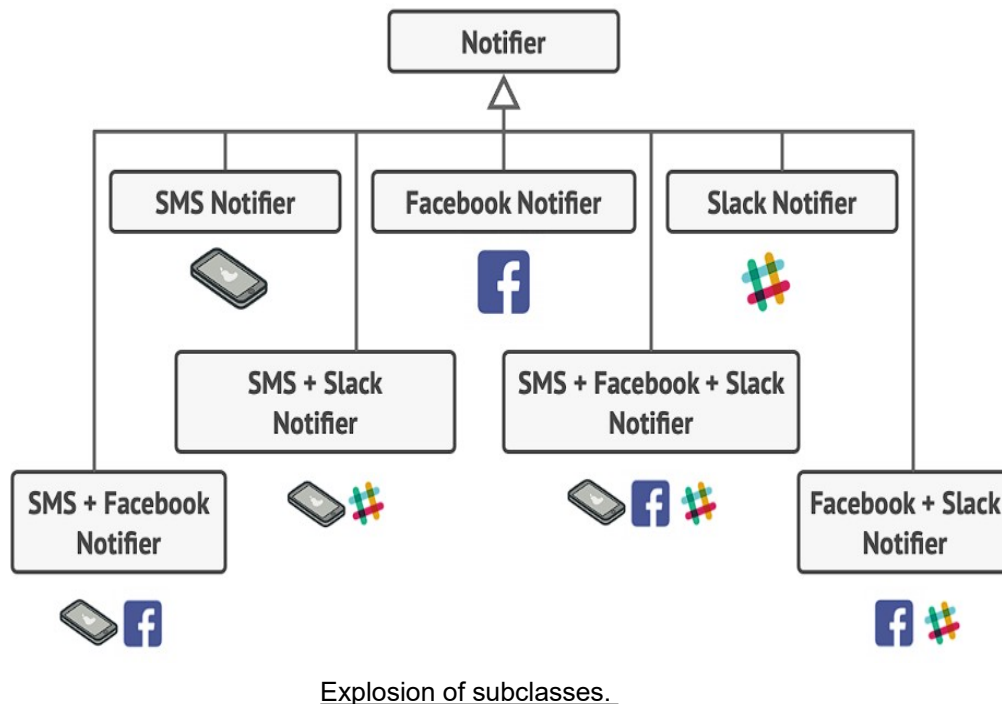
Users want SMS, Facebook, and Slack notifications. Extending the **Notifier** class with subclasses seems like a solution.



Each notification type is implemented as a notifier's subclass.

The Subclass Explosion

The demand for multiple notification types at once (e.g., email and SMS) leads to a combinatorial explosion of subclasses. This approach bloats code and makes maintenance difficult. A new strategy is needed to manage notification classes efficiently.



The Suggested Solution: Implementation of the Decorator Pattern

Instead of subclassing, the Decorator pattern “wraps” objects with other objects that add behavior. Each wrapper (or “decorator”) implements the same interface as the original object, so they’re interchangeable.

✓ Key Concepts:

- Wrapper Objects: Decorators wrap the original object and add behavior before or after delegating to it.
- Runtime Composition: You can stack decorators dynamically, allowing flexible behavior combinations.
- Interface Consistency: All decorators and the base object share the same interface, so client code doesn’t need to change.

•

Solution: Aggregation/Composition

Instead of inheritance, use aggregation or composition. One object "has a" reference to another and delegates work. Aggregation/Composition principle is key to the Decorator pattern.

Inheritance Limitations

- Static behavior: Cannot alter at runtime.
- Replaces objects: Requires creating new subclasses for changes.

Aggregation/Composition Benefits

- Dynamic behavior: Substitute linked objects at runtime.
- Flexible: Combine behaviors from various classes.

Delegation vs Inheritance

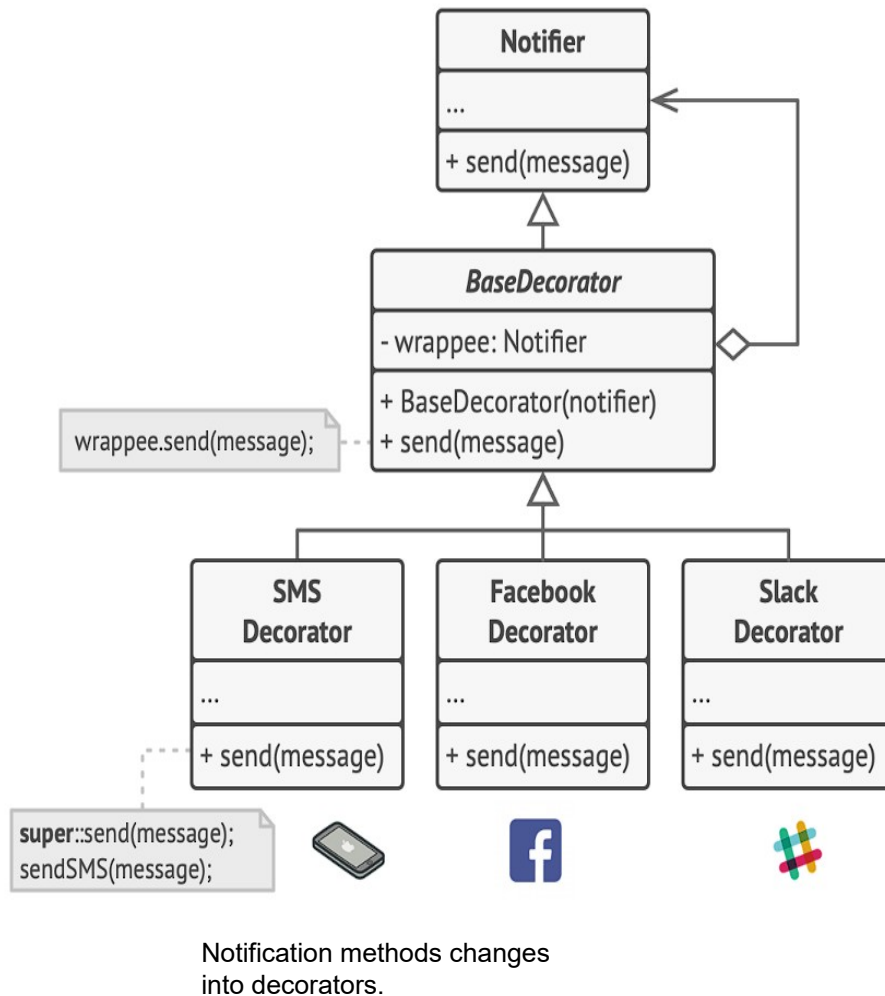
- Both composition and inheritance allow objects to perform tasks, but they differ in how behavior is assigned. With composition, one object holds a reference to another and delegates tasks to it. In contrast, inheritance embeds the behavior directly into the object via its superclass.
- This compositional approach enables you to swap out the referenced “helper” object at runtime, effectively changing the behavior of the main object on the fly. It also allows an object to tap into multiple behaviors by referencing several collaborators and delegating different responsibilities to each.

The Wrapper Concept in Decorator Pattern

- The term “wrapper” is often used to describe the Decorator pattern because it captures the essence of the design: a wrapper object surrounds a target object, mirrors its method set, and forwards incoming requests. However, it can also modify the outcome by adding logic before or after the delegation.
- A wrapper becomes a true decorator when it adheres to the same interface as the object it wraps. This ensures that, from the client’s point of view, both objects are interchangeable. By allowing the wrapper’s reference to accept any object conforming to that interface, you can layer multiple wrappers around a single object—each contributing its own behavior to the final result.

Decorator in Action: Notifications (1/2)

The base `Notifier` handles email. Other notification methods become decorators.



Core Idea: Instead of hardcoding every notification method (email, SMS, Slack, etc.) into one class or creating subclasses for each combination, you:

- Keep the basic functionality (e.g., email) in a core class (`Notifier`)
- Implement additional behaviors (e.g., SMS, Slack) as decorators that wrap the core

How It Works

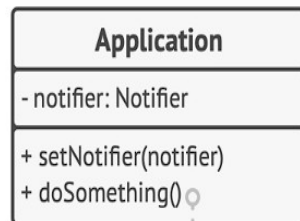
- Base Class: `Notifier` handles simple email notifications.
- Decorators: Each additional channel (SMS, Slack, etc.) is a separate class that wraps a `Notifier` or another decorator.
- Stacking: You wrap the base object in multiple decorators, forming a chain or stack of behaviors.

Decorator in Action: Notifications (2/2)

Clients wrap a basic notifier with decorators to match preferences, forming a stack of functionalities.

Client Transparency: Because all decorators implement the same interface, the client code doesn't need to know whether it's using a plain Notifier or a decorated one—it just calls the same methods.

```
stack = new Notifier()
if (facebookEnabled)
    stack = new FacebookDecorator(stack)
if (slackEnabled)
    stack = new SlackDecorator(stack)
app.setNotifier(stack)
```



```
notifier.send("Alert!")
// Email → Facebook → Slack
```

Apps configures complex stacks of notification decorators

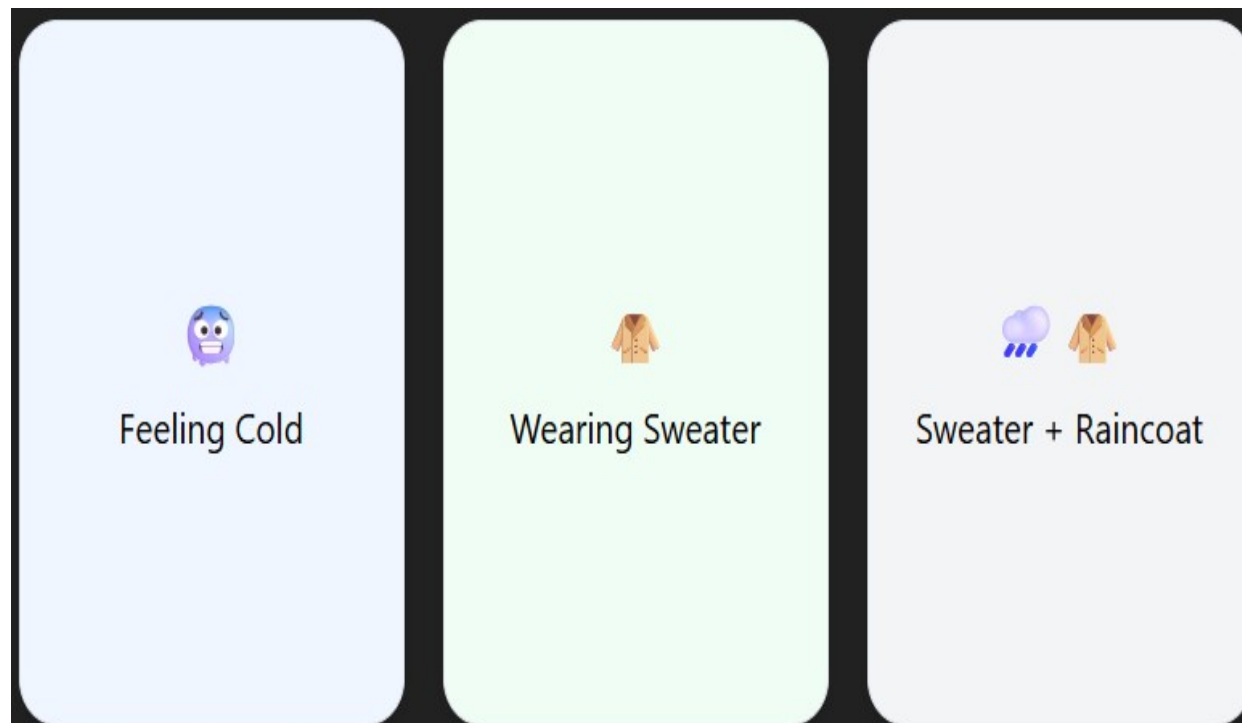
Extensibility

The Decorator pattern isn't limited to notifications. It can be applied to:

- Message formatting (e.g., MarkdownDecorator, HTMLDecorator)
- Recipient list composition (e.g., GroupDecorator, FilteredRecipientDecorator)
- As long as each decorator follows the same interface, it's possible to mix and match them freely.

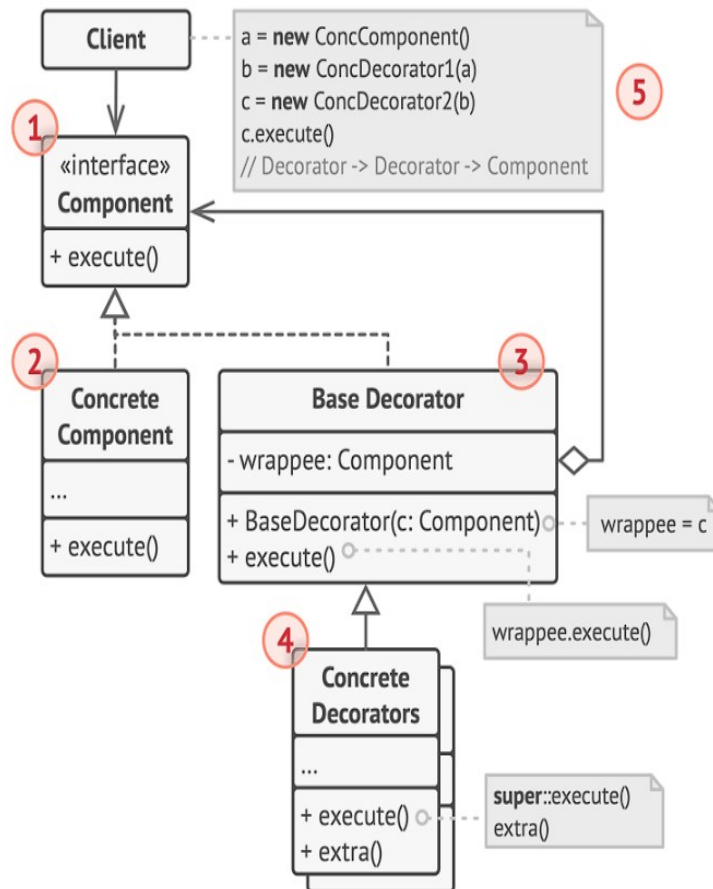
Real-World Analogy: Layering Clothes

Wearing clothes is like using decorators. You add layers (sweater, jacket, raincoat) to extend your basic behavior (staying warm, dry) without changing your core self. Each layer can be added or removed as needed.



Decorator Structure

Structure



1. Component

Common interface for wrappers and wrapped objects.

2. Concrete Component

The object being wrapped, defines base behavior.

3. Base Decorator

References a wrapped object, delegates all operations.

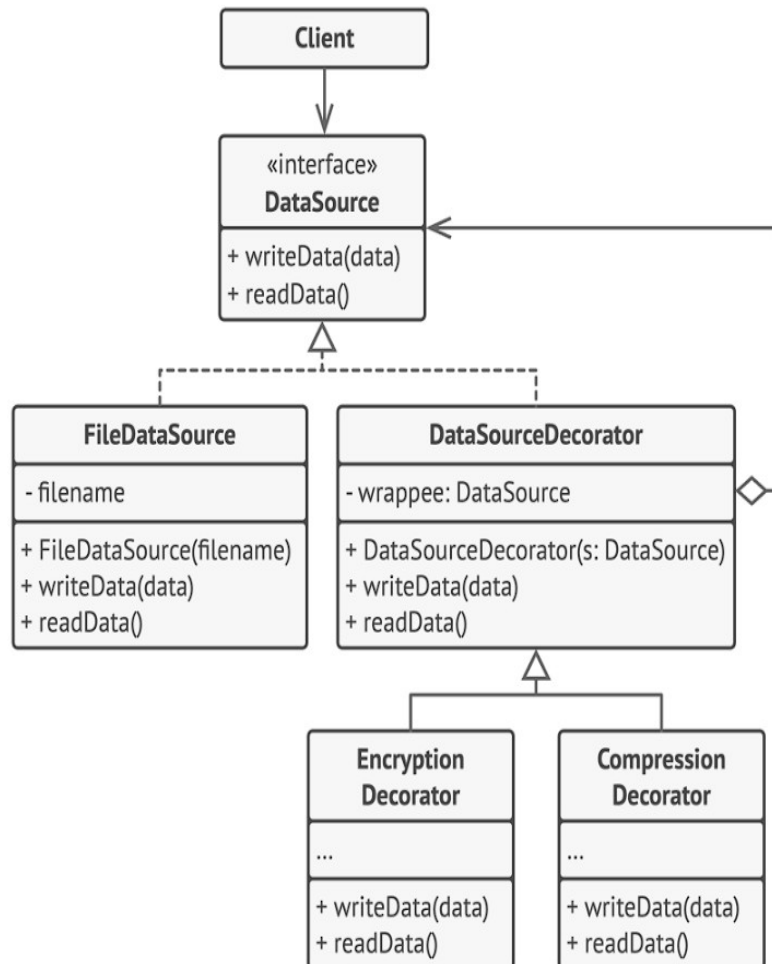
4. Concrete Decorators

Add extra behaviors dynamically, before or after parent calls.

5. Client

Composes decorators, interacts via the component interface.

Example



The Decorator pattern is used to add encryption and compression to data storage without modifying the core data-handling logic.

The application wraps the original data source with decorators that transform the data before writing (encrypting and compressing) and after reading (decompressing and decrypting).

Since all components share the same interface, they can be swapped or stacked seamlessly, keeping the client code clean and unaware of the underlying enhancements.

Java Code example

```
// The component interface defines operations that can be
// altered by decorators.
interface DataSource is
    method writeData(data)
    method readData():data

// Concrete components provide default implementations for the
// operations. There might be several variations of these
// classes in a program.
class FileDataSource implements DataSource is
    constructor FileDataSource(filename) { ... }

    method writeData(data) is
        // Write data to file.

    method readData():data is
        // Read data from file.

// The base decorator class follows the same interface as the
// other components. The primary purpose of this class is to
// define the wrapping interface for all concrete decorators.
// The default implementation of the wrapping code might include
// a field for storing a wrapped component and the means to
// initialize it.
class DataSourceDecorator implements DataSource is
    protected field wrappee: DataSource

    constructor DataSourceDecorator(source: DataSource) is
        wrappee = source

// The base decorator simply delegates all work to the
// wrapped component. Extra behaviors can be added in
// concrete decorators.
method writeData(data) is
    wrappee.writeData(data)

// Concrete decorators may call the parent implementation of
// the operation instead of calling the wrapped object
// directly. This approach simplifies extension of decorator
// classes.
method readData():data is
    return wrappee.readData()
```

```
// Concrete decorators must call methods on the wrapped object,
// but may add something of their own to the result. Decorators
// can execute the added behavior either before or after the
// call to a wrapped object.
class EncryptionDecorator extends DataSourceDecorator is
    method writeData(data) is
        // 1. Encrypt passed data.
        // 2. Pass encrypted data to the wrappee's writeData
        // method.

    method readData():data is
        // 1. Get data from the wrappee's readData method.
        // 2. Try to decrypt it if it's encrypted.
        // 3. Return the result.

// You can wrap objects in several layers of decorators.
class CompressionDecorator extends DataSourceDecorator is
    method writeData(data) is
        // 1. Compress passed data.
        // 2. Pass compressed data to the wrappee's writeData
        // method.

    method readData():data is
        // 1. Get data from the wrappee's readData method.
        // 2. Try to decompress it if it's compressed.
        // 3. Return the result.

// Option 1. A simple example of a decorator assembly.
class Application is
    method dumbUsageExample() is
        source = new FileDataSource("somefile.dat")
        source.writeData(salaryRecords)
        // The target file has been written with plain data.

        source = new CompressionDecorator(source)
        source.writeData(salaryRecords)
        // The target file has been written with compressed
        // data.

        source = new EncryptionDecorator(source)
        // The source variable now contains this:
        // Encryption > Compression > FileDataSource
        source.writeData(salaryRecords)
        // The file has been written with compressed and
        // encrypted data.
```

```
// Option 2. Client code that uses an external data source.
// SalaryManager objects neither know nor care about data
// storage specifics. They work with a pre-configured data
// source received from the app configurator.
class SalaryManager is
    field source: DataSource

    constructor SalaryManager(source: DataSource) { ... }

    method load() is
        return source.readData()

    method save() is
        source.writeData(salaryRecords)
        // ...Other useful methods...

// The app can assemble different stacks of decorators at
// runtime, depending on the configuration or environment.
class ApplicationConfigurator is
    method configurationExample() is
        source = new FileDataSource("salary.dat")
        if (enabledEncryption)
            source = new EncryptionDecorator(source)
        if (enabledCompression)
            source = new CompressionDecorator(source)

        logger = new SalaryManager(source)
        salary = logger.load()
        // ...
```

Source: <https://refactoring.guru/design-patterns/decorator>

Applicability & Implementation

When to Use

- Assign extra behaviors to objects at runtime.
- Structure business logic into optional layers.
- Extend behavior when inheritance is awkward or impossible.

How to Implement

- Identify primary component and optional layers.
- Define a common component interface.
- Create concrete component and base decorator classes.
- Implement concrete decorators extending base decorator.
- Client code handles decorator composition.

Pros and Cons

Pros

- Extend behavior without new subclasses.
- Add/remove responsibilities at runtime.
- Combine multiple behaviors.

Cons

- Hard to remove specific wrappers.
- Behavior can depend on decorator order.
- Initial configuration code can be complex.

The Decorator pattern offers flexibility but requires careful management of the wrapper stack.

Decorator vs Other Patterns (1/2)

1. Decorator vs Builder

- Decorator: Structural; wraps objects to add responsibilities dynamically.
- Builder: Creational; constructs complex objects step-by-step.

Key difference: Decorator modifies behavior at runtime, Builder organizes construction at creation time.

2. Decorator vs Factory Method

- Decorator: Focuses on extending functionality without subclassing.
- Factory Method: Focuses on delegating object creation to subclasses.

Key difference: Decorator is about behavioral extension, Factory Method is about object instantiation.

Decorator vs Other Patterns (2/2)

3. Decorator vs Abstract Factory

- Decorator: Adds features to a single object instance.
- Abstract Factory: Creates families of related objects without specifying concrete classes.

Key difference: Decorator enhances individual objects, Abstract Factory ensures consistent product families.

4. Decorator vs Adapter

- Decorator: Preserves the interface, adds new behavior.
- Adapter: Changes the interface, makes incompatible classes work together.

Key difference: Decorator = same interface, more features. Adapter = different interface, compatibility.

5. Decorator vs Bridge

- Decorator: Wraps objects to add responsibilities dynamically.
- Bridge: Decouples abstraction from implementation so both can vary independently.

Key difference: Decorator is about layering features, Bridge is about separating dimensions of change.

Decorator vs adapter: deep-dive differences (1/3)

Both are structural patterns, but they solve different problems. Decorator augments behavior without changing the interface. Adapter transforms an interface so incompatible components can collaborate. Understanding that intent keeps your architecture clean and avoids mixing responsibilities.

Core intent and impact

- Problem focus: Decorator adds responsibilities dynamically to an object while preserving its interface. Adapter converts one interface into another so existing code can be reused without modification.
- Interface handling: Decorator keeps the same interface (is-a Component). Adapter presents a different interface (is-a Target; has-an Adaptee).
- Client perspective: With Decorator, the client calls the same methods but experiences richer behavior. With Adapter, the client uses a familiar Target API while the adapter translates calls to a different Adaptee API.
- Primary value: Decorator prioritizes composable feature layering (logging, caching, encryption). Adapter prioritizes compatibility and reuse (legacy API, third-party library).
- Typical use cases: Decorator: stream pipelines, GUI embellishments, cross-cutting concerns. Adapter: legacy system integration, protocol normalization, vendor API unification.

Tricky pitfalls and signals

Overdecoration risk: Excessive decorator stacks can obscure behavior and complicate debugging. Establish naming, tracing, and ordering rules.

Adapter leakage risk: If Target exposes semantics the Adaptee can't truly provide, the adapter becomes leaky or fragile. Validate capability parity and error mapping.

Coupling signals: Decorators should depend on the Component interface only; don't peek into concrete types. Adapters should depend on Target and Adaptee interfaces, not concrete classes, to remain replaceable.

Decorator vs adapter: deep-dive differences (2/3)

Java example: adapter

Goal: Your application expects a `PaymentProcessor` with `process(amount)`, but a legacy gateway exposes `charge(total, currency)`. The adapter translates between them.

```
// Target interface (what the app expects)
public interface PaymentProcessor {
    boolean process(double amount);
}

// Adaptee (legacy or third-party API)
public class LegacyPaymentGateway {
    public boolean charge(double total, String currencyCode) {
        System.out.println("Charging " + total + " " + currencyCode);
        return true;
    }
}

// Adapter (wraps Adaptee to match Target)
public class PaymentAdapter implements PaymentProcessor {
    private final LegacyPaymentGateway gateway;
    private final String defaultCurrency;

    public PaymentAdapter(LegacyPaymentGateway gateway,
        String defaultCurrency) {
        this.gateway = gateway;
        this.defaultCurrency = defaultCurrency;
    }

    @Override
    public boolean process(double amount) {
        // Translate to the Adaptee's API
        return gateway.charge(amount, defaultCurrency);
    }
}
```

```
// Client
public class CheckoutService {
    private final PaymentProcessor processor;

    public CheckoutService(PaymentProcessor processor) {
        this.processor = processor;
    }

    public void checkout(double total) {
        if (!processor.process(total)) {
            throw new IllegalStateException("Payment failed");
        }
        System.out.println("Payment succeeded");
    }

    public static void main(String[] args) {
        LegacyPaymentGateway gateway = new LegacyPaymentGateway();
        PaymentProcessor processor = new PaymentAdapter(gateway, "USD");
        new CheckoutService(processor).checkout(99.99);
    }
}
```

Decorator vs adapter: deep-dive differences (3/3)

Java example: decorator. Goal: You have a DataSource with write/read. Add compression and encryption without changing client code by stacking decorators.

```
// Component interface
public interface DataSource {
    void write(byte[] data);
    byte[] read();
}

// Concrete component
public class FileDataSource implements DataSource {
    private final java.nio.file.Path path;

    public FileDataSource(String filename) {
        this.path = java.nio.file.Paths.get(filename);
    }

    @Override
    public void write(byte[] data) {
        try {
            java.nio.file.Files.write(path, data);
        } catch (java.io.IOException e) {
            throw new RuntimeException(e);
        }
    }

    @Override
    public byte[] read() {
        try {
            return java.nio.file.Files.readAllBytes(path);
        } catch (java.io.IOException e) {
            throw new RuntimeException(e);
        }
    }
}

// Base decorator
public abstract class DataSourceDecorator implements DataSource {
    protected final DataSource wrappee;

    protected DataSourceDecorator(DataSource wrappee) {
        this.wrappee = wrappee;
    }

    @Override
    public void write(byte[] data) { wrappee.write(data); }

    @Override
    public byte[] read() { return wrappee.read(); }
}

// Concrete decorators
public class CompressionDecorator extends DataSourceDecorator {
    public CompressionDecorator(DataSource wrappee) {
        super(wrappee);
    }
}
```

```
@Override
public void write(byte[] data) {
    byte[] compressed = compress(data);
    super.write(compressed);
}

@Override
public byte[] read() { return decompress(super.read()); }

private byte[] compress(byte[] input) {
    // Naive demo compression (replace with real algorithm)
    return java.util.Base64.getEncoder().encode(input);
}

private byte[] decompress(byte[] input) {
    return java.util.Base64.getDecoder().decode(input);
}

public class EncryptionDecorator extends DataSourceDecorator {
    private final javax.crypto.SecretKey key;

    public EncryptionDecorator(DataSource wrappee,
        javax.crypto.SecretKey key) {
        super(wrappee);
        this.key = key;
    }

    @Override
    public void write(byte[] data) { super.write(encrypt(data)); }

    @Override
    public byte[] read() { return decrypt(super.read()); }

    private byte[] encrypt(byte[] input) {
        try {
            var cipher = javax.crypto.Cipher.getInstance("AES");
            cipher.init(javax.crypto.Cipher.ENCRYPT_MODE, key);
            return cipher.doFinal(input);
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }

    private byte[] decrypt(byte[] input) {
        try {
            var cipher = javax.crypto.Cipher.getInstance("AES");
            cipher.init(javax.crypto.Cipher.DECRYPT_MODE, key);
            return cipher.doFinal(input);
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
}
```

```
// Client composes behavior
public class Demo {
    public static void main(String[] args) throws Exception {
        javax.crypto.KeyGenerator kg =
            javax.crypto.KeyGenerator.getInstance("AES");
        kg.init(128);
        var key = kg.generateKey();

        DataSource ds = new
            FileDataSource("store.bin");
        ds = new CompressionDecorator(ds);
        ds = new EncryptionDecorator(ds, key);

        ds.write("Hello, Observer!".getBytes());
        byte[] data = ds.read();
        System.out.println(new String(data)); // "Hello,
        Observer!"
    }
}
```

Decision guidance

Use adapter when the obstacle is interface mismatch. It's about compatibility and reuse without changing the adaptee.

Use decorator when the goal is to enrich behavior modularly. It's about layering features transparently and flexibly.

Combine them carefully:

Adapter first to normalize an interface, then Decorator to add cross-cutting features (e.g., logging, retries, caching).

Avoid placing adapters inside deep decorator stacks unless necessary; normalize once, then decorate.

Use of Decorator Pattern with other Patterns

Patterns Decorator Can Combine With

Factory Method / Abstract Factory → Factories can produce decorated objects directly (e.g., a factory returns a `LoggingService` wrapped in a `CachingDecorator`).

Builder → Builders can assemble objects with optional decorators (e.g., building a pipeline of decorated data sources).

Adapter → After adapting an interface, decorators can add cross-cutting concerns (e.g., logging, retries, monitoring).

Bridge → Bridge separates abstraction from implementation, and decorators can then add runtime features to either side.

Practical Use Cases

I/O Streams in Java: `BufferedInputStream`, `DataInputStream`, `ZipInputStream` are classic decorators.

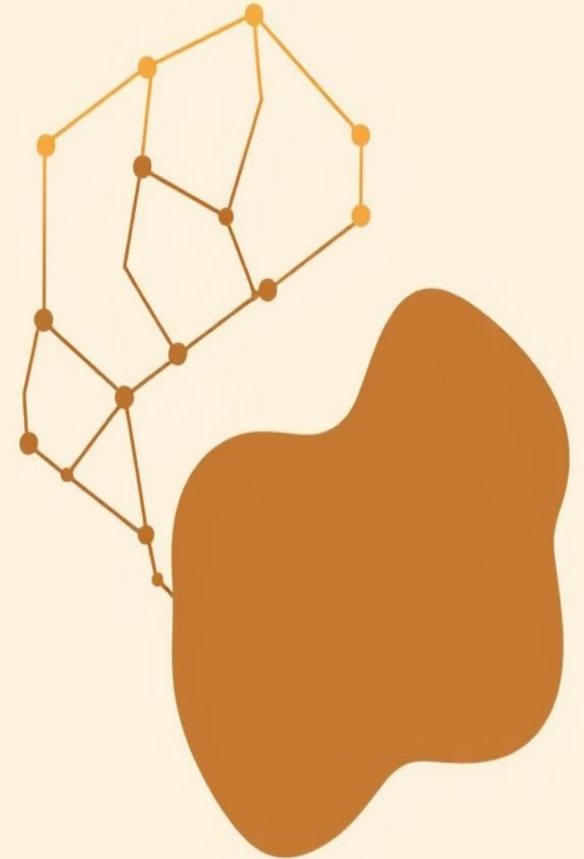
Web Middleware: Wrapping HTTP requests/responses with logging, authentication, or compression decorators.

Service Layer: A `PaymentProcessor` decorated with `LoggingDecorator`, `RetryDecorator`, `SecurityDecorator`.

Combined with Factory: A `ServiceFactory` that always returns a decorated service instance with monitoring enabled.

Chapter 7, Clean Code -Error Handling

While often seen as a necessary evil, effective error handling is paramount for building reliable and maintainable software systems. We'll explore techniques to ensure your code gracefully manages unexpected situations without sacrificing readability or logic.



The Essence of Clean Error Handling

Error handling is an unavoidable aspect of programming. Input can be abnormal, devices can fail, and unexpected situations can arise. As programmers, we are responsible for ensuring our code behaves correctly even when things go wrong. However, the challenge lies in integrating error handling without obscuring the core logic of our applications.

Error handling is important, but if it obscures logic, it's wrong.

Many codebases are unfortunately dominated by scattered error handling, making it nearly impossible to discern the primary function of the code. This chapter will outline techniques and considerations to write code that is both clean and robust, handling errors with grace and style, ensuring your applications are both functional and understandable.

Embrace Exceptions: Beyond Return Codes

In the early days of programming, languages often lacked robust exception mechanisms. Developers relied on error flags or return codes, which callers had to manually check immediately after each function call. This approach, while functional, led to cluttered and less readable code, as illustrated in the traditional `DeviceController.java` example.

```
public class DeviceController {
    public void sendShutDown() {
        DeviceHandle handle = getHandle(DEV1);

        // Check the state of the device
        if (handle != DeviceHandle.INVALID) {

            // Save the device status to the record field
            retrieveDeviceRecord(handle);

            // If not suspended, shut down
            if (record.getStatus() != DEVICE_SUSPENDED) {
                pauseDevice(handle);
                clearDeviceWorkOueue(handle);
                closeDevice(handle);
            }

            else {
                logger.log("Device suspended. Unable to shut down");
            }
        }

        else {
            logger.log("Invalid handle for: " + DEV1.toString());
        }
    }
}
```

The primary issue with return codes is the burden they place on the caller. Forgetting a single error check can lead to unpredictable behavior. Exceptions, however, offer a cleaner alternative by separating error detection from normal program flow. When an error occurs, an exception is thrown, allowing the calling code's logic to remain focused and uncluttered.

Exceptions for Clarity: A Refactored Example

By leveraging exceptions, we can significantly improve the readability and maintainability of our code. The following example demonstrates how the `DeviceController` can be refactored to use exceptions, leading to a clearer separation of concerns between the shutdown algorithm and error handling.

```
public class DeviceController {  
  
    // ...  
    public void sendShutDown() {  
        try {  
            tryToShutDown();  
        } catch (DeviceShutDownError e) {  
            logger.log(e);  
        }  
    }  
  
    private void tryToShutDown() throws DeviceShutDownError {  
  
        DeviceHandle handle = getHandle(DEV1);  
        DeviceRecord record = retrieveDeviceRecord(handle);  
        pauseDevice(handle);  
        clearDeviceWorkQueue(handle);  
        closeDevice(handle);  
    }  
    DeviceHandle getHandle(DeviceID id) {  
  
        // ...  
        throw new  
        DeviceShutDownError("Invalid handle for: " + id.toString());  
    }  
}
```

Notice the improved clarity. The `tryToShutDown()` method now focuses solely on the device shutdown logic, while the `sendShutDown()` method handles any potential `DeviceShutDownError`. This separation allows developers to understand each concern independently, making the code easier to debug and modify. This isn't just about aesthetics; it's about creating a more robust and understandable system.

Designing with Exceptions: Try-Catch-Finally First

Exceptions define a crucial scope within your program. When code executes within a `try` block, you're essentially stating that execution can be aborted at any point, with control then resuming in the corresponding `catch` block. This makes `try` blocks akin to transactions, where the `catch` block must ensure the program remains in a consistent state, regardless of what went wrong.

A best practice is to start by writing your `try-catch-finally` statement first when dealing with code that might throw exceptions. This approach helps define the expected behavior for the user of that code, ensuring consistency even in the face of errors. Let's consider an example of accessing a file to read serialized objects:

```
@Test(expected = StorageException.class)
public void retrieveSectionShouldThrowOnInvalidFileName()
{
    sectionStore.retrieveSection("invalid-file");
}

public List<RecordedGrip> retrieveSection(String sectionName)
{
    try {
        FileInputStream stream = new FileInputStream(sectionName);
        stream.close();
    }
    catch (FileNotFoundException e) {
        throw new StorageException("retrieval error", e);
    }

    return new ArrayList<RecordedGrip>();
}
```

By establishing the `try-catch` structure early, we define the transaction scope. We can then use Test-Driven Development (TDD) to build the remaining logic within the `try` block, confident that error handling is already in place. This method encourages building robust error handling from the ground up, maintaining the transactional nature of the code.

Unchecked Exceptions and Contextual Information

The debate surrounding checked vs. unchecked exceptions has long been a topic in Java development. While checked exceptions were initially seen as beneficial for enforcing error handling, their cost in terms of code coupling and maintenance has become evident. Languages like C#, Python, and Ruby demonstrate that robust software can be built without them.

The primary "price" of checked exceptions is an Open/Closed Principle violation. A change in a low-level method that throws a new checked exception can force signature changes across many higher-level methods, leading to a cascade of modifications and recompilations. This breaks encapsulation, as functions in the call path become aware of low-level error details they shouldn't need to know.

Recommendation:

For general application development, prefer unchecked exceptions. They reduce coupling and allow for more flexible error handling at appropriate levels of abstraction. Checked exceptions can be useful in critical libraries where callers absolutely must handle specific errors.

Furthermore, every exception thrown should provide sufficient context to diagnose the error. While a stack trace is helpful, it doesn't convey the intent of the failed operation. Always create informative error messages that describe the operation that failed and the type of failure. If logging, ensure enough information is passed to log the error effectively.

Defining Exception Classes by Caller Needs

When classifying errors and defining exception classes, the most crucial consideration should be how they are caught. Classifying by source or type (e.g., device failure, network failure) can lead to overly granular and redundant exception handling, especially when the recovery action is similar across different error types.

Consider a scenario with a third-party library that throws multiple distinct exceptions for similar underlying issues. A naive approach might lead to duplicated catch blocks:

```
ACMEPort port = new ACMEPort(12);
try { port.open();}

catch (DeviceResponseException e)
{ reportPortError(e); logger.log("Device response exception", e);}

catch (ATM1212UnlockedException e) { reportPortError(e); logger.log("Unlock exception", e);}

catch (GMXError e) { reportPortError(e);
logger.log("Device response exception");}

finally { // ...}
```

This duplication indicates a problem. If the action taken in response to these exceptions is largely the same (e.g., reporting an error and ensuring continued operation), we can simplify by wrapping the third-party API and translating its diverse exceptions into a common, application-specific exception type.

The Power of API Wrapping and Special Case Pattern

Wrapping third-party APIs is a powerful best practice. It minimizes your dependencies on external libraries, making it easier to switch implementations in the future. It also simplifies testing by allowing you to mock out third-party calls. Crucially, it gives you control over the API design, letting you define an interface that aligns with your application's needs, including its error handling.

```
public class LocalPort { private ACMEPort innerPort;

public LocalPort(int portNumber)
{ innerPort = new ACMEPort(portNumber); }

public void open() {
try { innerPort.open(); }
catch (DeviceResponseException | ATM1212UnlockedException | GMXError e)
{ throw new PortDeviceFailure(e); } } // ...}
```

In this example, `LocalPort` acts as a wrapper, catching various `ACMEPort` exceptions and re-throwing them as a single `PortDeviceFailure`. This allows the client code to handle a single, more meaningful exception type, leading to cleaner and more consistent error management.

This concept extends to the Special Case Pattern [Fowler]. This pattern involves creating a class or configuring an object to handle a special case, thereby encapsulating exceptional behavior and preventing the client code from needing to deal with it directly. For instance, instead of throwing an exception when no meal expenses are found, an `ExpenseReportDAO` could return a `MealExpenses` object that defaults to a per diem amount:

```
public class PerDiemMealExpenses implements MealExpenses { public int getTotal() { // return the per
diem default }}
```

This approach allows the main logic to remain clean and focused on the normal flow, as the "exceptional" case is handled transparently by the special case object.

Avoiding Nulls: A Path to Robustness

Two common practices that invite errors and clutter code are returning null and passing null.

The pervasive need for null checks can quickly dominate a codebase, making it difficult to read and prone to `NullPointerExceptions` (NPEs).

Don't Return Null

Returning null forces callers to constantly check for null, creating boilerplate code and increasing the risk of missed checks. Instead, consider throwing an exception or, even better, returning a Special Case object. For collections, returning an empty list (e.g., `Collections.emptyList()` in Java) is a much cleaner alternative to returning null:

```
// Bad:
List<Employee> employees = getEmployees();
if (employees != null) { for(Employee e : employees) {    totalPay += e.getPay(); }}
// Good:
public List<Employee> getEmployees() { if( .. there are no employees .. )    return
Collections.emptyList(); // ...}
```

Don't Pass Null

Passing null into methods is even more problematic, as it often leads to immediate NPEs within the called method. While assertions can document expectations, they don't prevent runtime errors. The most rational approach is to forbid passing null by default. This allows you to code with the assurance that arguments are valid, significantly reducing careless mistakes.

```
public class MetricsCalculator {
    public double xProjection(Point p1, Point p2) {

        // Explicit null check and exception
        if (p1 == null || p2 == null) {
            throw new IllegalArgumentException("Invalid argument for MetricsCalculator.xProjection");    }

        // Or, better, rely on caller to not pass null
        return (p2.x - p1.x) * 1.5;    }}

```

By minimizing nulls, your code becomes inherently more robust and easier to reason about, as you eliminate an entire class of common runtime errors.

Conclusion: The Synergy of Clean and Robust Code

As we've explored, clean code is not merely about readability; it's also about robustness. These two goals are not conflicting but complementary. By consciously separating error handling concerns from our main business logic, we can achieve both.



Separate Concerns

Treat error handling as a distinct concern, allowing your core logic to remain clean and focused on its primary algorithm.



Leverage Exceptions

Use exceptions over return codes to keep calling code clean and ensure errors are handled at the appropriate level.



Provide Context

Ensure exceptions carry informative messages and context to aid in debugging and error resolution.



Avoid Nulls

Minimize returning or passing nulls to prevent `NullPointerException`s and simplify code logic.

When we can reason about our error handling independently, we make significant strides in the maintainability and reliability of our software. This approach not only makes your code easier to understand but also more resilient to unexpected situations, a hallmark of professional software engineering.

Further Reading:

- [Martin]: *Agile Software Development: Principles, Patterns, and Practices*, Robert C. Martin, Prentice Hall, 2002.
- [Fowler]: *Pattern-Oriented Software Architecture, Volume 2: Patterns for Concurrent and Networked Objects*, Douglas C. Schmidt, Michael Stal, Hans Rohnert, Frank Buschmann, John Wiley & Sons, 2000.